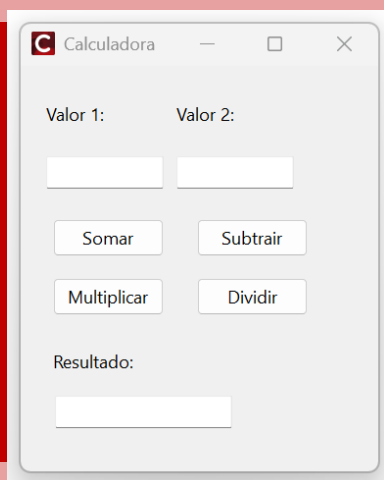




RAD Studio | C++Builder 13

FLORENCE



MÓDULO 1 - BÁSICO

PROJETO FUNÇÕES BÁSICAS C++ BUILDER

Criado por: Thiago Santos
MVP Embarcadero. 2026

SUMÁRIO

SUMÁRIO 2

Introdução ao Curso de C++Builder 13 Florence	3
O que você vai aprender neste curso	4
Sobre o instrutor	5
Conhecendo as funções básicas na prática	6
CONVERSÃO DE TIPOS DE DADOS	6
MANIPULAÇÃO DE STRINGS E LISTA DE STRINGS	13
TRATAMENTO DE EXCEÇÃO	21

Seja bem-vindo!!

Introdução ao Curso de C++Builder 13 Florence

Este curso foi desenvolvido para apresentar o **C++Builder 13 Florence**, a versão mais recente da plataforma de desenvolvimento da Embarcadero, com foco em quem deseja aprender **C++ de forma prática, produtiva e aplicada ao desenvolvimento de aplicações reais**.

Ao longo desta formação, o aluno será introduzido aos principais conceitos do **C++ funcional aplicado ao C++Builder**, entendendo não apenas a linguagem, mas também como utilizá-la de forma eficiente dentro de um ambiente RAD (Rapid Application Development), que acelera o processo de criação de software sem *abrir mão de desempenho e controle*.

A importância de utilizar o C++Builder 13 Florence

Utilizar a versão mais recente do C++Builder é fundamental para garantir **compatibilidade com tecnologias atuais**, maior estabilidade da IDE e acesso aos recursos mais modernos da linguagem C++. O C++Builder 13 Florence traz melhorias significativas no compilador baseado em Clang, suporte aprimorado ao padrão moderno do C++, além de uma IDE mais rápida, segura e alinhada às necessidades do mercado atual.

Aprender com uma versão atual evita retrabalho futuro, reduz problemas de compatibilidade e prepara o aluno para atuar em projetos profissionais, corporativos e comerciais, utilizando ferramentas reconhecidas e amplamente adotadas por empresas.

O que você vai aprender neste curso

Este curso tem como objetivo fornecer uma **base funcional e prática**, permitindo que o aluno desenvolva aplicações reais desde os primeiros módulos. Entre os principais conhecimentos adquiridos, destacam-se:

- Fundamentos do C++ aplicados ao C++Builder
- Estrutura da IDE e fluxo de desenvolvimento
- Criação de aplicações visuais utilizando VCL
- Manipulação de eventos e componentes
- Lógica de programação aplicada a projetos reais
- Integração básica com banco de dados
- Boas práticas para desenvolvimento em C++Builder

Ao final do curso, o aluno terá conhecimento suficiente para **criar aplicações funcionais**, entender projetos existentes e evoluir para níveis mais avançados da linguagem e da ferramenta.

Por que escolher o C++Builder

O C++Builder se diferencia por unir o **alto desempenho do C++** com a **produtividade do desenvolvimento visual**. Ele permite criar aplicações robustas, rápidas e profissionais com muito menos código do que abordagens tradicionais, mantendo total controle sobre a linguagem e o desempenho. Além disso, o C++Builder é amplamente utilizado em sistemas corporativos, aplicações industriais, softwares comerciais e soluções que exigem **performance, estabilidade e longevidade**. Aprender C++Builder é investir em uma tecnologia madura, sólida e ainda extremamente relevante no mercado. Este curso é o primeiro passo para quem deseja dominar o **C++ moderno aplicado ao desenvolvimento profissional**, utilizando uma das ferramentas mais completas e produtivas disponíveis atualmente.

Está pronto? Vamos começar!!

Sobre o instrutor

Meu nome é **Thiago Santos** e sou profissional da área de tecnologia, atuando há vários anos com **desenvolvimento de software, análise de negócios e treinamentos técnicos**, com foco na linguagem **C++** e na plataforma **C++Builder**.

Ao longo da minha trajetória profissional, tive a oportunidade de trabalhar diretamente com projetos reais, ambientes corporativos e soluções que exigem **alto desempenho, estabilidade e manutenção a longo prazo**. Essa experiência prática me permitiu entender não apenas a linguagem C++, mas também como aplicá-la de forma eficiente no dia a dia de empresas e equipes de desenvolvimento.

Sou **MVP da Embarcadero na linguagem C++**, reconhecimento concedido a profissionais que contribuem ativamente para a comunidade, compartilham conhecimento e utilizam as tecnologias Embarcadero em projetos reais. Além disso, atuo como **instrutor, agente de negócios e consultor técnico**, auxiliando empresas e desenvolvedores na adoção e evolução do C++Builder em seus sistemas.

Neste curso, meu objetivo não é apenas ensinar comandos ou conceitos isolados, mas **transmitir a forma correta de pensar, estruturar e desenvolver aplicações em C++Builder**, com base em experiências reais de mercado. Todo o conteúdo foi planejado para ser direto, prático e aplicável, evitando excesso de teoria desconectada da prática.

Acredito que aprender programação vai muito além de escrever código: envolve entender processos, tomar decisões técnicas corretas e criar soluções que realmente funcionem. É exatamente essa visão que compartilho ao longo deste curso.

Seja bem-vindo(a) à formação. Espero que este conteúdo contribua de forma significativa para sua evolução profissional e técnica no desenvolvimento com **C++Builder**.

Meus contatos e plataformas estão disponíveis:

<https://sam-cpp.wordpress.com/>

Conhecendo as funções básicas na prática

CONVERSÃO DE TIPOS DE DADOS

Durante o desenvolvimento de uma aplicação em **C++Builder**, é comum trabalhar com diferentes tipos de valores fornecidos pelo usuário, como **texto, números inteiros, valores decimais ou monetários, datas, horas e valores lógicos**.

O C++Builder disponibiliza diversos **tipos de dados** e **funções auxiliares** para manipular esses valores corretamente. Para isso, o desenvolvedor deve conhecer os tipos disponíveis e realizar as conversões adequadas, garantindo que os dados sejam utilizados corretamente na semântica do código.

A seguir estão os **principais tipos de dados mais utilizados em aplicações C++Builder**:

String (UnicodeString)

No C++Builder, o tipo mais utilizado para textos é o `UnicodeString`.

```
UnicodeString Nome;
```

Este tipo é capaz de armazenar textos inseridos pelo usuário, aceitando:

- Caracteres alfanuméricos;
 - Caracteres especiais;
 - Suporte completo a Unicode (acentos e caracteres internacionais);
- É amplamente utilizado em componentes visuais como `TEdit`, `TLabel`, `TMemo`, entre outros.

Integer (int):

O tipo `int` é utilizado para armazenar **valores numéricos inteiros**.

```
Int Idade;
```

Características:

- Aceita apenas números inteiros
- Não aceita letras ou caracteres especiais
- Muito utilizado para contadores, índices e cálculos simples.

Real (double / float / Currency):

Para valores numéricos com casas decimais ou valores monetários, o C++Builder utiliza tipos como:

```
double Valor;  
float Taxa;  
Currency Preco;
```

Características:

- Permite números fracionários
- Pode ser utilizado para cálculos financeiros
- *Currency* é recomendado para valores monetários, pois evita erros de precisão

Boolean (bool):

O tipo **bool** armazena valores lógicos:

```
Bool Ativo;
```

Valores possíveis:

- true (verdadeiro)
- false (falso)

É amplamente utilizado para:

- Controle de fluxo (if, while, for)
- Estados de componentes
- Configuração de propriedades

OBSERVAÇÃO SOBRE OUTROS TIPOS

Além dos tipos apresentados, o C++Builder possui diversos outros tipos específicos que variam conforme:

- Tamanho em memória
- Intervalo de valores
- Finalidade específica (datas, horas, ponteiros, enums, etc.)

Esses tipos são utilizados para **otimizar desempenho, economizar memória** ou **tratar dados especiais**.

CONVERSÃO DE TIPOS DE DADOS (CASTING)

Ao manipular dados, é comum a necessidade de **converter valores de um tipo para outro**, especialmente quando se trabalha com entradas de usuário, que normalmente chegam como texto (UnicodeString).

No C++Builder, essas conversões são realizadas através de **funções auxiliares**, como:

```
int StrToInt(UnicodeString);  
double StrToFloat(UnicodeString);  
UnicodeString IntToStr(int);  
UnicodeString FloatToStr(double);
```

Esses métodos facilitam o **casting de dados**, garantindo segurança e clareza no código.

EXEMPLO PRÁTICO

Para exemplificar a conversão de tipos de dados de forma simples, crie um novo projeto no **C++Builder** com o nome **“Calculadora”** e modele o formulário conforme ilustrado na Figura 1:

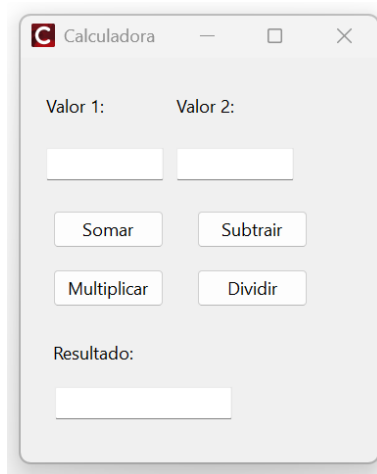


Figura 1: Formulário com as funções matemáticas básicas

No botão “Somar”, a princípio, tente escrever o seguinte código e executar a aplicação:


```

//-----
void __fastcall TForm1::btnSomarClick(TObject *Sender)
{
    int Valor1;
    int Valor2;

    Valor1 = editValor1->Text;
    Valor2 = editValor2->Text;
    editResultado->Text = Valor1 + Valor2;
}
//-----

```

Código 1

Ao tentar compilar o projeto, o C++Builder irá acusar uma inconsistência na semântica do código acima com a seguinte mensagem:

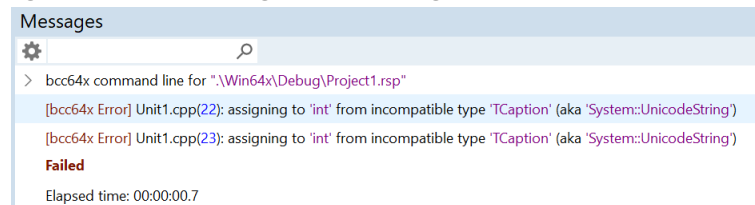


Figura 2: Error Insight

Este tipo de mensagem ocorre quando há uma tentativa de atribuir valores de diferentes tipos de dados. A mensagem acima, por exemplo, faz uma crítica ao tentar atribuir um tipo de dado *string* a uma variável do tipo *integer*. Para que esta atribuição de valores ocorra, é necessário utilizar alguns métodos para converter os tipos de dados, conforme o código abaixo:

```

//-----
void __fastcall TForm1::btnSomarClick(TObject *Sender)
{
    int Valor1;
    int Valor2;

    Valor1 = StrToInt(editValor1->Text);
    Valor2 = StrToInt(editValor2->Text);
    editResultado->Text = IntToStr(Valor1 + Valor2);
}
//-----

```

Código 2

Note que nas linhas de código após declarar as variáveis *Valor1* e *Valor2*, o método *StrToInt* foi utilizado para converter um valor do tipo *string* para uma variável do tipo *integer*, enquanto na terceira linha ocorre o oposto, isto é, a soma dos dois valores é convertida para *string* (texto) e atribuída ao campo “edtResultado”.

O exemplo acima demonstra que para realizar um cálculo de soma, é preciso primeiramente converter os dados digitados para números. Se esta conversão não for realizada, os valores inseridos serão simplesmente concatenados ao invés de calculados, por se tratarem de tipo de dados *string*. O código a seguir demonstra uma comparação dos resultados e a importância de utilizar a conversão:

```

//-----
void __fastcall TForm1::btnSomarClick(TObject *Sender)
{
    UnicodeString Texto;
    int Numero;

    Texto = editValor1->Text + editValor2->Text;
    Numero = StrToInt(editValor1->Text) + StrToInt(editValor2->Text);
    edtResultado->Text = IntToStr(Numero);
}
//-----

```

Código 3

Se o valor “4” for digitado no campo “editValor1.Text”, e o valor “8” digitado no campo “editValor2.Text”, os resultados serão:

Texto = 48 (concatenação de 4 e 8)

Numero = 12 (soma de 4 + 8)

O próximo passo é escrever o código para o botão de subtração de valores. O procedimento é basicamente o mesmo que a função de soma, incluindo as mesmas conversões de tipos de dados:

```

//-----
void __fastcall TForm1::btnSubtrairClick(TObject *Sender)
{
    int Valor1;
    int Valor2;

    Valor1 = StrToInt(editValor1->Text);
    Valor2 = StrToInt(editValor2->Text);
    edtResultado->Text = IntToStr(Valor1 - Valor2);
}
//-----

```

Código 4

Para os botões de multiplicação e divisão, os valores inseridos nos campos poderão ser fracionados em casas decimais separados por uma vírgula. Neste caso, é incorreto utilizar variáveis do tipo *int* e os métodos *StrToInt* e *IntToStr*. Se estes forem usados, os números das casas decimais serão ignorados e o resultado será arredondado para um número inteiro. Para trabalhar com números fracionários, o tipo de dados ***double*** deve ser utilizado nas funções dos botões, conforme seguem os códigos a seguir:

```

//-----
void __fastcall TForm1::btnMultiplicarClick(TObject *Sender)
{
    double Valor1, Valor2;
40     Valor1 = StrToFloat(editValor1->Text);
    Valor2 = StrToFloat(editValor2->Text);
43     edtResultado->Text = FloatToStr(Valor1 * Valor2);
}
//-----

```

Código 5

```

//-----
void __fastcall TForm1::btnDividirClick(TObject *Sender)
{
49     double Valor1, Valor2;
50     Valor1 = StrToFloat(editValor1->Text);
    Valor2 = StrToFloat(editValor2->Text);
    edtResultado->Text = FloatToStr(Valor1 / Valor2);
}
//-----

```

Código 6

Verifique no código acima a utilização dos métodos *StrToFloat* e *FloatToStr* para a conversão de dados entre *string* e *double*. Esta conversão é importante principalmente na função de divisão, visto que o resultado pode ser um número fracionado. Execute a aplicação, insira alguns valores nos campos de entrada e faça alguns testes utilizando os quatro botões implementados.

Além dos exemplos acima, as conversões de tipos de dados também podem ocorrer com valores de datas e horas. No mesmo projeto, insira dois componentes *DateTimePicker* no formulário e altere a propriedade *Name* do primeiro para “dtpData1” e do segundo para “dtpData2”. No evento *onChange* do componente “dtpData2”, escreva o código abaixo:

```

//-----
void __fastcall TForm1::dtpData2Change(TObject *Sender)
{
59     dtpData1->Date = dtpData2->Date;
60 }
//-----

```

Código 7

Execute a aplicação, selecione uma data no componente “dtpData2”, e verifique que a data selecionada será automaticamente atribuída ao componente “dtpData1”. Note que neste caso não foi necessário realizar nenhuma conversão de dados, já que as propriedades dos dois componentes compartilham o mesmo tipo de dado. Em seguida, adicione um botão ao formulário para que ele dispare uma mensagem com a data selecionada no campo “dtpData2”:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
64     ShowMessage(dtpData2->Date);
}
//-----

```

Código 8

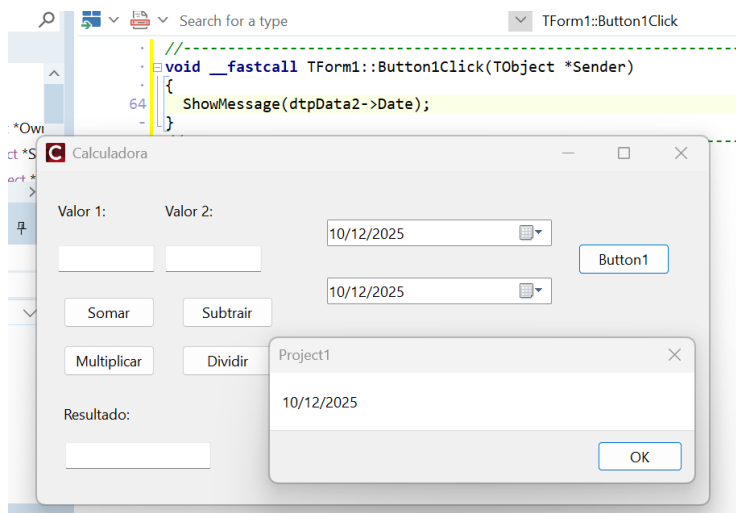


Figura 3: Compiler Messages

Em C++Builder, este código **pode compilar, PORÉM:**

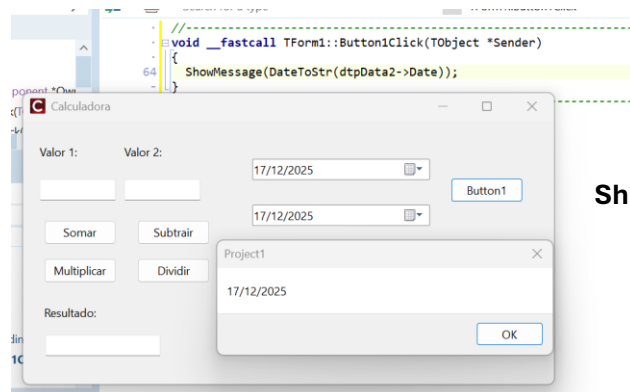
Isso NÃO é conversão automática correta.

Isso acontece porque:

- TDateTime em C++Builder é um typedef de double
- O compilador C++ é menos restritivo
- O método ShowMessage recebe um UnicodeString
- O compilador tenta usar conversões implícitas, o que:
 - Pode gerar warning
 - Pode gerar comportamento indefinido
 - Não é uma boa prática
 -

Ou seja: não é que esteja “certo”, apenas o C++ permite passar — **mas o valor exibido pode ser incorreto.**

O método *correto* é este:



ShowMessage (DateToStr (dtpData2->Date));

MANIPULAÇÃO DE STRINGS E LISTA DE STRINGS

Sem dúvidas, o tipo de dado **UnicodeString** é amplamente utilizado no desenvolvimento de aplicações em **C++Builder**, pois representa textos em geral. Por esse motivo, o C++Builder disponibiliza diversas **funções e métodos** para tratamento e manipulação de strings, permitindo operações como conversão de maiúsculas e minúsculas, concatenação, busca, substituição e manipulação de listas de strings.

Crie um novo projeto no C++Builder (VCL Forms Application), adicione um componente *TMemo* e um *TRadioGroup* no formulário, alterando as propriedades *Name* para “MemoTexto” e “RadioGroupTexto”, respectivamente. No componente *RadioGroup*, clique no botão de reticências da propriedade *Items* para abrir o editor e adicione as opções “Maiúsculo” e “Minúsculo”, separadas por uma quebra de linha. Deixe com uma coluna para os itens no *RadioGroup* fiquem lado a lado. Organize os componentes no formulário de modo que o visual fique parecido com a Figura 4:

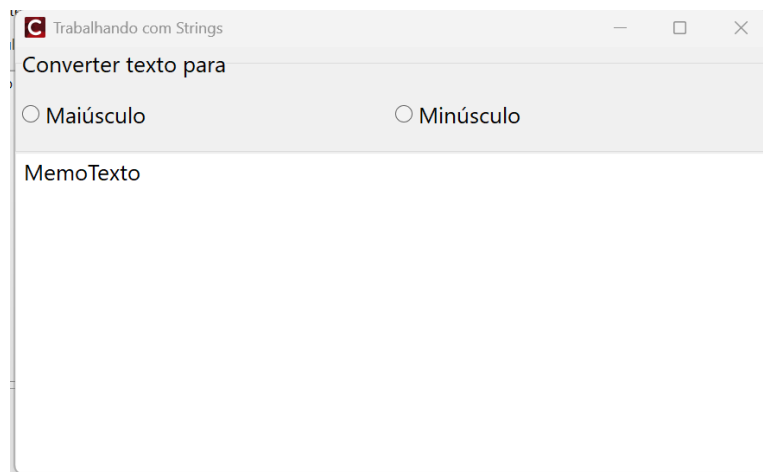


Figura 4: Tela para conversão de texto

Como se pode notar, este é um exemplo para converter um texto para maiúsculo ou minúsculo, conforme a opção selecionada. O código a seguir deve ser inserido no evento *OnClick* do *RadioGroup*, para que o texto seja modificado no *Memo*:

```
//-----
void __fastcall TForm1::RadioGroupTextoClick(TObject *Sender)
{
    if (RadioGroupTexto->ItemIndex == 0)
    {
        MemoTexto->Lines->Text = UpperCase(MemoTexto->Lines->Text);
        MemoTexto->CharCase = ecUpperCase;
    }
    else if (RadioGroupTexto->ItemIndex == 1)
    {
        MemoTexto->Lines->Text = LowerCase(MemoTexto->Lines->Text);
        MemoTexto->CharCase = ecLowerCase;
    }
}
//-----
```

Código 11

Execute a aplicação, digite algum texto no *Memo* e então clique nas opções para conferir o resultado. Os métodos *UpperCase* e *LowerCase* tem a funcionalidade de converter todo o conteúdo string de um componente para maiúsculo ou minúsculo, enquanto a propriedade *CharCase* define que os caracteres digitados no componente estarão de acordo com a opção selecionada.

Em continuidade ao exemplo acima, adicione mais um componente *Memo* e um componente *Button* no formulário, nomeando-os como “MemoCopia” e “ButtonCopiar”, respectivamente. O objetivo agora é copiar o conteúdo do “MemoTexto” para o “MemoCopia”, que basicamente pode ser realizado com o código abaixo:

```
MemoCopia->Lines->Text = MemoTexto->Lines->Text;
```

O procedimento acima copia o conteúdo perfeitamente, mas a intenção é copiar apenas uma parte do “MemoTexto”, e não o texto completo. Para que isso seja possível, deve-se utilizar o método **Text.Substring** e indicar a quantidade de caracteres que devem ser copiados. Escreva o código a seguir no botão “Copiar” e substitua os parâmetros conforme desejado. No caso do código abaixo, somente as vinte primeiras letras serão copiadas para o “MemoCopia”:

```
//-----
void __fastcall TForm1::ButtonCopiarClick(TObject *Sender)
{
    MemoCopia->Lines->Text =
35 MemoTexto->Lines->Text.SubString(1, 20);
}
//-----
```

Código 12

O primeiro parâmetro do método *Text.Substring* deve receber o texto a ser copiado, enquanto o segundo parâmetro indica o índice de início da cópia (iniciando-se pelo 1), e o terceiro a quantidade de caracteres que será copiada. Execute a aplicação mais uma vez, insira um conteúdo no *Memo* e clique no botão “Copiar”, conforme a Figura 5:

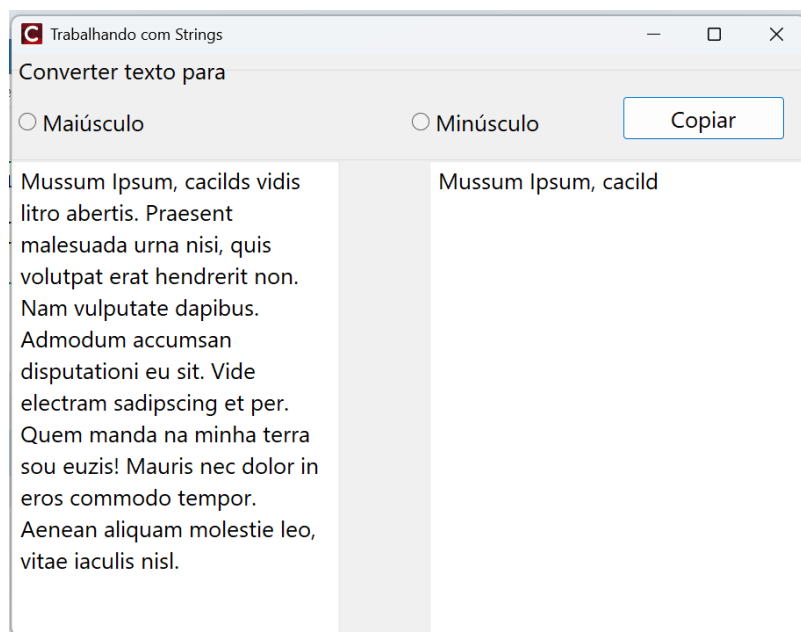


Figura 5: Copiando uma parte de um texto

A única objeção do código acima é a limitação da cópia na quantidade de caracteres especificada na função *Text.Substring*. O ideal, portanto, seria copiar um texto selecionado pelo próprio usuário através da função *SelectText*. Entre novamente no evento do botão “Copiar” e substitua o código pelas linhas abaixo:

```
//-----  
void __fastcall TForm1::ButtonCopiarClick(TObject *Sender)  
{  
    MemoCopia->Lines->Text =  
35    MemoTexto->SelText.SubString(1, MemoTexto->SelText.Length());  
}  
//-----
```

Código 13

Já que não é possível saber a quantidade de caracteres que serão selecionados em tempo de projeto, o último parâmetro da função *Copy* deve ser obtido em tempo de execução. Note na utilização do método *Length*, que retorna a quantidade de caracteres de uma string, ou seja, o texto selecionado no “MemoTexto”. Ao executar a aplicação com o mesmo texto da Figura 5, obtém-se o seguinte resultado:

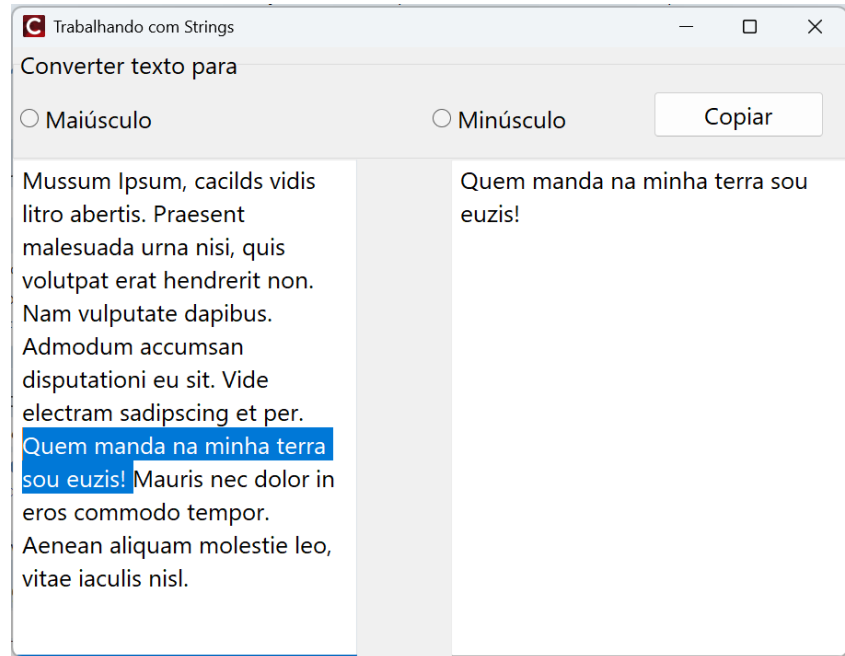
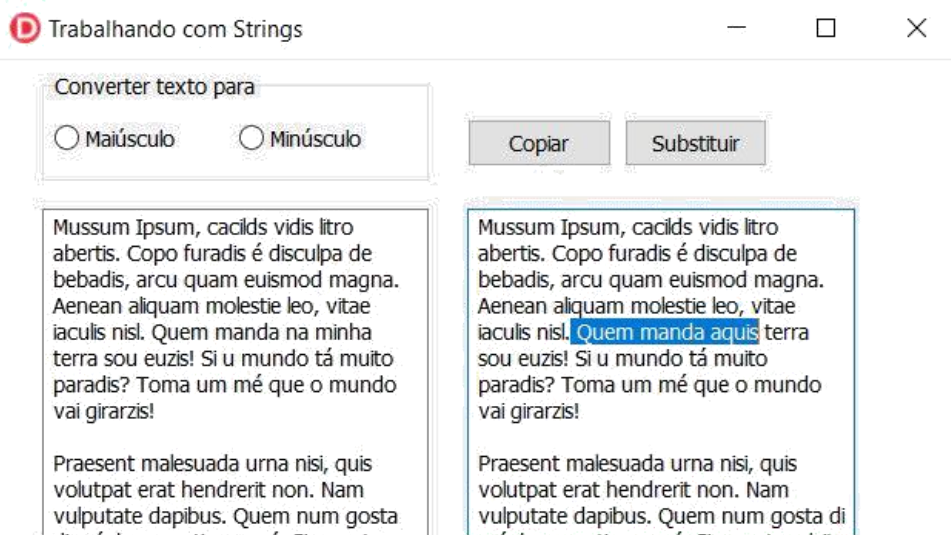


Figura 6: Copiando apenas o texto selecionado

O próximo passo é criar uma função para substituir determinadas palavras do texto. Adicione mais um componente *Button* ao formulário e altere a propriedade *Caption* para “Substituir”. A função deste botão será solicitar um texto a ser procurado no “MemoTexto”, em seguida solicitar o texto para substituição, e então atribuir o resultado ao componente “MemoCopia”. Para obter os textos digitados pelo usuário, a função *InputBox* será executada duas vezes. O *InputBox* consiste em uma pequena caixa de diálogo para a inserção de um valor qualquer, e retorna este valor para uma variável específica no código. Por fim, o texto será substituído por meio da utilização da função *StringReplace*, conforme o código abaixo do botão “Substituir”:




```

//-----
void __fastcall TForm1::btnSubstituirClick(TObject *Sender)
{
    String TextoProcurar, TextoSubstituir;

    TextoProcurar = InputBox(" Digite o texto a ser encontrado",
        "Texto a ser encontrado: ", "");

    TextoSubstituir = InputBox("Digite o texto para substituir",
        "Substituir por: ", "");

    MemoCopia->Lines->Text =
        StringReplace(
50  MemoTexto->Lines->Text, //Texto a ser tratado
        TextoProcurar, //Texto a ser procurado
        TextoSubstituir, //Texto para substituição
        TReplaceFlags() << rfReplaceAll << rfIgnoreCase); //opções de substituição
}
//-----

```

Código 14

O último parâmetro da função *StringReplace* consiste em configurar as opções de substituição do texto. No código acima, a opção *rfReplaceAll* foi adicionada para substituir todas as ocorrências do texto encontrado, enquanto a opção *rfIgnoreCase* determina que as diferenças entre maiúsculas e minúsculas serão ignoradas. A Figura 7 demonstra um exemplo da substituição de texto utilizando o botão “Substituir”:

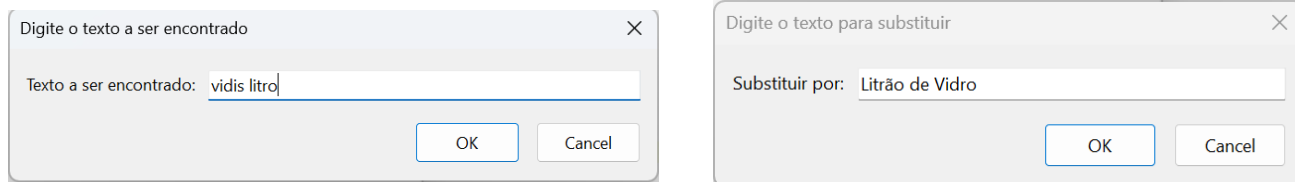


Figura 7: “vidis litro” por “Litrão de Vidro”;

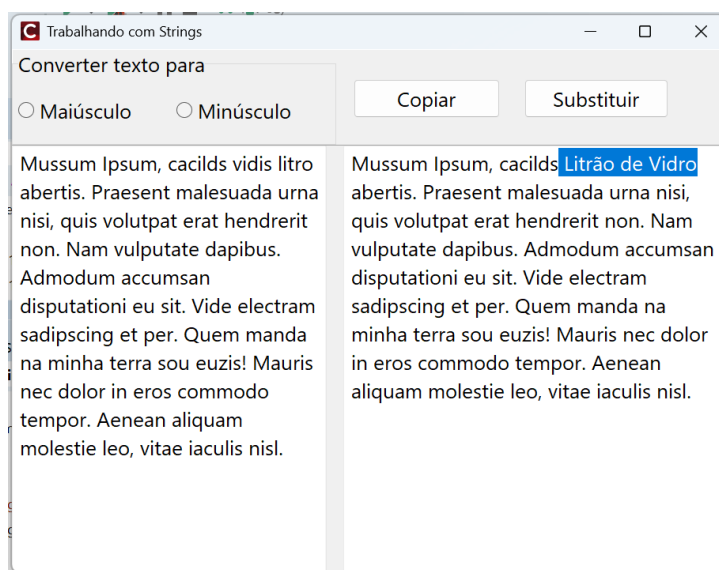


Figura 8: Substituindo uma palavra do texto

Em determinadas situações pode ser necessário trabalhar com várias strings em um mesmo método, como separar palavras de um texto, por exemplo. O C++Builder permite que um conjunto de strings seja armazenado em uma mesma variável denominada “Lista de Strings”, semelhante a um vetor de textos. Dessa forma, cada string dentro dessa lista é acessada por um índice correspondente à sua posição, iniciando-se do índice zero.

Crie um novo projeto e modele o visual de modo que fique semelhante à Figura 8, composto dos componentes *Edit*, *Button* e *ListBox*.

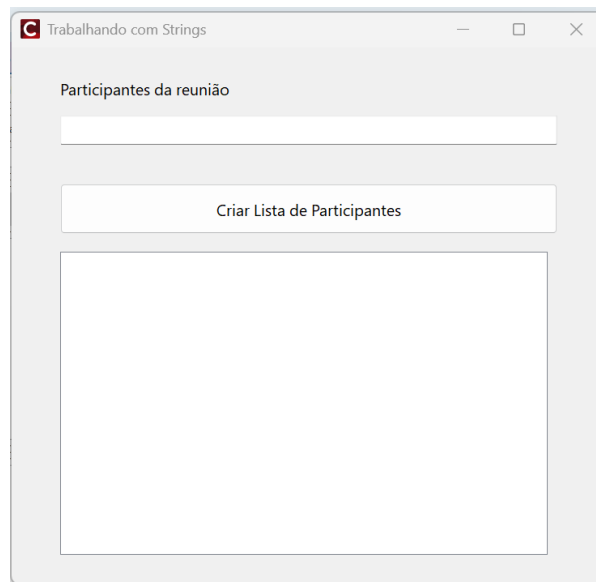


Figura 8: Trabalhando com Lista de Strings

A função do botão “Criar Lista de Participantes” será separar os nomes inseridos no campo “EditParticipantes” e enumerá-los dentro do campo “ListBoxNomes”. Para desenvolver esta funcionalidade, uma Lista de Strings será criada e tratada através do método *CommaText*, que consiste na separação de valores baseados no caractere de vírgula (*comma*). Portanto, os nomes inseridos no campo “EditParticipantes” deverão ser separados por vírgula para que a Lista seja criada de forma correta. O botão “Criar Lista de Participantes” deverá conter o seguinte código:

```
//-----
void __fastcall TForm1::btnCriarListaClick(TObject *Sender)
{
    TStringList *Lista = new TStringList(); // Em C++Builder, classes da VCL devem
                                           // ser criadas com 'new'.

    try
    {
        // Atribui ao TStringList o texto digitado no EditParticipantes.
        // A propriedade CommaText interpreta o texto separado por vírgulas
        // e converte automaticamente cada item em uma linha da Lista.
        Lista->CommaText = EditParticipantes->Text;

        // Copia todos os itens da TStringList para a coleção Items do ListBox.
        ListBoxNomes->Items->Assign(Lista);
    }
    finally
    {
        // Libera a memória ocupada pela TStringList.
        delete Lista;
    }
}
//-----
```

Código 15

Execute a aplicação e insira alguns nomes no campo “EditParticipantes” separados por vírgula. Em seguida, clique no botão “Criar Lista de Participantes” e confira o resultado no ListBox. A Figura 9 exibe um exemplo com uma inserção de cinco nomes:

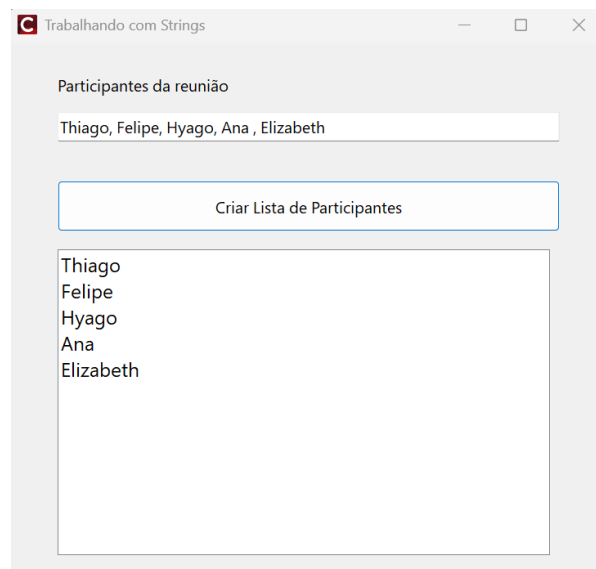


Figura 9: Valores atribuídos à Lista de Strings

A exibição dos nomes dos participantes pode ficar ainda mais interessante se forem enumerados ao clicar no botão e ordenados por ordem alfabética. Para que isto ocorra, o código acima será modificado para que os índices da lista sejam manipulados por meio de uma variável e um laço de repetição. Note também na utilização do método *Sort* para a ordenação dos valores da Lista de Strings:

```

void __fastcall TForm1::btnCriarListaClick(TObject *Sender)
{
    TStringList *Lista = new TStringList(); // Em C++Builder, classes da VCL devem
                                           // ser criadas com 'new'.
20
21    try
22    {
        // Atribui ao TStringList o texto digitado no EditParticipantes.
        // A propriedade CommaText interpreta o texto separado por vírgulas
        // e converte automaticamente cada item em uma linha da lista.
        Lista->CommaText = EditParticipantes->Text;
        Lista->Sort(); // Ordena os nomes por ordem alfabética;

        for (int indice = 0; indice < Lista->Count; indice++)
        {
30            Lista->Strings[indice] =
                IntToStr(indice + 1) + " - " + Lista->Strings[indice];
        }

        // Copia todos os itens da TStringList para a coleção Items do ListBox.
        ListBoxNomes->Items->Assign(Lista);
    }
    finally
    {
40        // Libera a memória ocupada pela TStringList.
        delete Lista;
    }
}

```

Código 16

No C++Builder, o laço de repetição é controlado pela quantidade de itens existentes na lista, a qual é preenchida a partir dos nomes informados no componente **EditParticipantes**. O laço inicia no índice zero e é executado enquanto o índice for menor que o total de elementos da lista (`Lista->Count`). Dessa forma, todos os itens são percorridos corretamente, sem a necessidade de subtrair uma unidade do valor total, uma vez que a condição de parada do `for` já é exclusiva. A Figura 10 apresenta um exemplo utilizando a ordenação dos itens da lista.

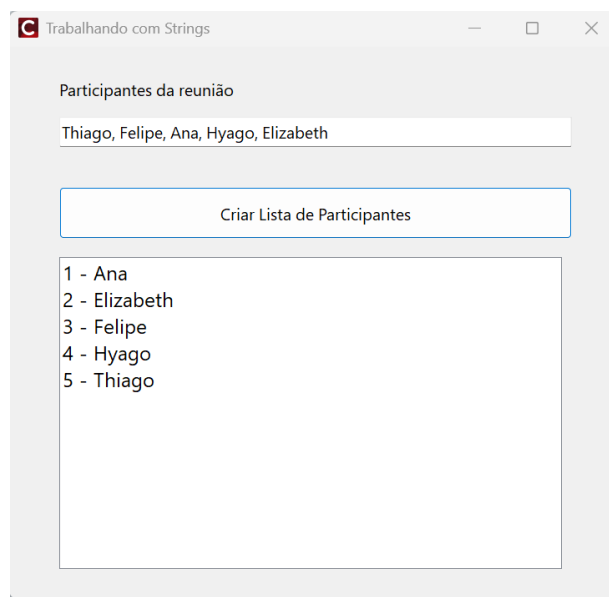


Figura 10: Manipulando e ordenando os índices da StringList

TRATAMENTO DE EXCEÇÃO

Mesmo que o desenvolvedor passe horas testando uma aplicação em busca de erros, sempre é possível que a aplicação apresente inconsistências quando for utilizada na prática em um ambiente de trabalho. Estes erros são resultados de uma falha na

semântica, evento inesperado no sistema ou inserção de valores inconsistentes, e são conhecidos como “exceções” dentro da programação.

Com base neste conhecimento, os desenvolvedores procuram controlar os erros de uma aplicação em um processo chamado “tratamento de exceção”, que consiste em exibir uma mensagem de erro informativa ou desviar o fluxo de instruções de um método caso ocorra algum erro.

Crie um novo projeto no C++Builder, insira um componente *Button* e três componentes *Edit* para realizar uma simples soma de dois valores. Renomeie os três componentes *Edit* para “EditValor1”, “EditValor2” e “EditResultado”. O visual do formulário com os componentes inseridos deve ficar semelhante à Figura 11:

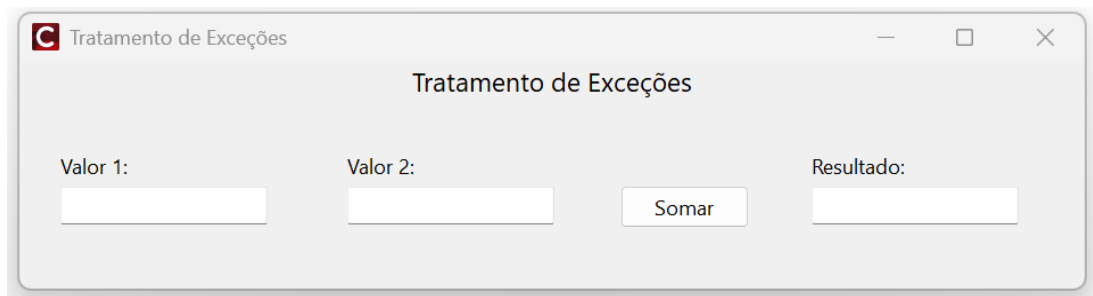


Figura 11: Tela para exemplificar o tratamento de exceção

No botão “Somar”, escreva o código a seguir:

```
//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    int Valor1, Valor2;
    Valor1 = StrToInt(EditValor1->Text);
    Valor2 = StrToInt(EditValor2->Text);
    EditResultado->Text = IntToStr(Valor1 + Valor2);
}
//-----
```

Código 17

Execute a aplicação e verifique que o resultado é calculado corretamente quando dois números são inseridos nos campos. Porém, tente inserir a letra “A” no primeiro campo, “B” no segundo campo, e então clicar no botão “Somar”. O Delphi lançará a seguinte exceção, indicando que a letra “A” não corresponde a um valor numérico:

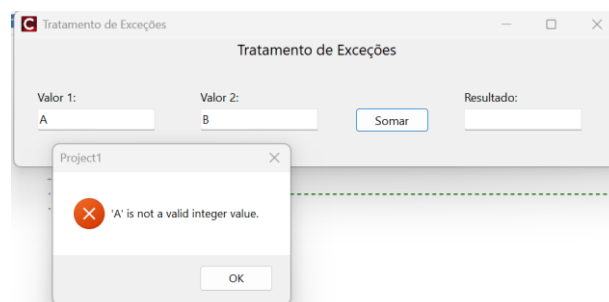


Figura 12: Exceção gerada ao tentar atribuir um valor *string* a uma variável *int*

Esta exceção deve ser tratada para evitar que a aplicação tente fazer a soma de valores não-numéricos e inibir esta mensagem, que pode ser um tanto quanto desconhecida para o usuário. Para tratar exceções no C++Builder, utiliza-se as palavras **try e catch** para circundar o código que pode gerar o erro. Substitua o código do botão “Somar” pelas linhas abaixo:

```
//
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    int Valor1, Valor2;

    try
    {
        Valor1 = StrToInt(EditValor1->Text);
        Valor2 = StrToInt(EditValor2->Text);

        EditResultado->Text = IntToStr(Valor1 + Valor2);
    }
    catch (...)
    {
        MessageDlg(
            "Digite apenas números nos campos!",
            mtWarning,
            TMsgDlgButtons() << mbOK,
            0
        );
    }
}
//
```

Código 18

O código depois da palavra *try* “tentará” ser lido e executado com sucesso, caso contrário o fluxo de instruções será direcionado para o código dentro da palavra *catch*. Portanto, execute a aplicação e tente realizar o teste novamente, inserindo “A” e “B” nos campos. O C++Builder encontrará uma exceção logo na primeira linha de código, onde a conversão da letra “A” para tipo inteiro não é permitida. Assim, a mensagem contida dentro do *catch* será exibida, conforme a Figura 13:

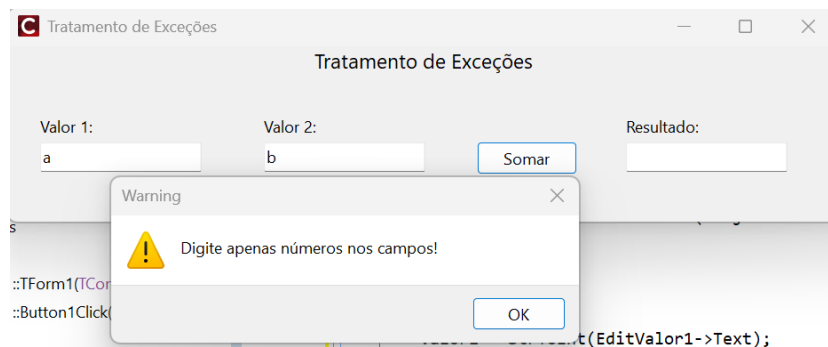


Figura 13: Tratando uma exceção dentro da aplicação

Se houver mais componentes *Edit* no formulário, talvez seja melhor personalizar o código para exibir a mensagem referente ao componente que está gerando a exceção em questão. Para isso, basta inserir o *try* e *catch* para validar o valor de cada componente distinto e executar uma saída do método utilizando o comando *return*, caso algum erro seja encontrado. Altere mais uma vez o código do botão “Somar” de acordo com o código abaixo:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    int Valor1, Valor2;

    try
    {
        Valor1 = StrToInt(EditValor1->Text);
    }
    catch (...)
    {
        MessageDlg(
            "O primeiro valor digitado é inválido.",
            mtWarning,
            TMsgDlgButtons() << mbOK,
            0
        );
        return; // equivalente ao Exit do Delphi
    }

    try
    {
        Valor2 = StrToInt(EditValor2->Text);
    }
    catch (...)
    {
        MessageDlg(
            "O segundo valor digitado é inválido.",
            mtWarning,
            TMsgDlgButtons() << mbOK,
            0
        );
        return; // equivalente ao Exit do Delphi
    }

    EditResultado->Text = IntToStr(Valor1 + Valor2);
}
//-----

```

Código 19

Além do try e catch , existe também o bloco __finally. A função desta palavra é executar um conjunto de instruções independente se houver ou não uma exceção na aplicação. Para exemplificar o uso do __finally, modifique o código do botão Somar inserindo as seguintes linhas:

```

//-----
void __fastcall TForm1::Button1Click(TObject *Sender)
{
    int Valor1, Valor2;
    try
    {
        try
        {
            Valor1 = StrToInt(EditValor1->Text);
            Valor2 = StrToInt(EditValor2->Text);
            EditResultado->Text = IntToStr(Valor1 + Valor2);
            ShowMessage("O resultado da soma é: " + EditResultado->Text);
        }
        catch (...) {
            MessageDlg("Digite apenas valores numéricos.", mtWarning, TMsgDlgButtons() << mbOK, 0);
        }
    }
    finally
    {
        EditValor1->Clear();
        EditValor2->Clear();
        EditResultado->Clear();
        EditValor1->SetFocus();
    }
}
//-----

```

Código 20

Conclusão

O uso de try, catch e __finally garante que a aplicação seja mais robusta, evitando falhas inesperadas e proporcionando uma melhor experiência ao usuário.